CONTENTS

10주차 모델을 어떻게 내보내고, 무엇으로 통과시키는가

1. 오늘의 질문

2. 이 장에서 실행할 것

시작 전 준비

3. 배치와 온라인은 속도가 아니라 실패의 형태로 갈린다

4. 서빙 API는 모델에 신원을 붙이고 입력 규격을 집행한다

5. shadow는 실패 반경을 0으로 두고 후보를 재평가한다

6. 게이트는 층으로 쌓고, 동일성은 바이트로 확인한다

7. 핵심 요약

8. 과제

9. 더 알아보기

10주차 모델을 어떻게 내보내고, 무엇으로 통과시키는가

> **10주차 · 모델 배포와 CI/CD**
>
> 이 장의 수치는 모두 `practice/chapter10/`의 코드를 실행해 얻은 값이다. 예시를 위해 지어낸 숫자는 없다. 인용한 값의 출처 파일은 출력 블록 첫 줄에 적었다.
>
> 한 가지 예외가 있다. 컨테이너(container)를 띄워 얻은 값은 이 강의 환경에서 Docker 데몬을 켤 수 없어 다시 돌리지 못했고, **과거 실행이 남긴 산출물에서 옮긴 값**이다. 그 블록의 출처는 `practice/chapter10/data/output/`으로 적었다.

☐ 9주차에서 champion 별칭까지 붙은 모델은 아직 실습 스크립트 안에서만 실행됨

- ○ 이용자는 스크립트를 실행하지 않으므로, 모델은 외부 시스템이 호출할 수 있는 상태로 올려야 함
- ○ 올리는 일과, 올려도 되는지 판정하는 일이 이 장의 내용임

☐ 이 장도 설명과 실습을 나누지 않음

- ○ 절마다 실행할 명령이 나오고 그 아래에 실제 출력과 해석이 이어짐
- ○ 순서대로 실행하면 이 장의 수치를 직접 생성함

1. 오늘의 질문

☐ 예측을 미리 만들어 둘 것인가, 요청이 도착한 순간 계산할 것인가?

☐ 서빙 API가 “지금 무엇을 서빙하고 있는가”에 답하지 못하면 무엇을 잃는가?

☐ 새 모델 후보를 이용자에게 위험을 주지 않고 실제 요청 위에서 평가할 수 있는가?

☐ 같은 확인을 왜 여러 층에서 반복하는가?

2. 이 장에서 실행할 것

☐ 모델을 내보내는 일은 확인 다섯 가지로 나뉘고, 각각 다른 절에서 실행함

확인할 것	어떻게 확인하는가	다루는 절
앱이 입력 규격과 응답 규격을 지키는가	서버를 띄우지 않고 앱 함수를 직접 호출해 요청 7건을 통과시킴	4
실제 프로세스로 띄워도 기동되는가	호스트에서 서빙 프로세스를 띄워 같은 호출을 보냄	6
환경이 바뀌어도 같은 답을 내는가	컨테이너에서 같은 호출을 반복하고 두 답을 대조함	6
지금 무엇을 서빙 중인지 답할 수 있는가	응답에 모델 이름·버전·로드 출처 세 필드를 함께 싣게 함	4
새 후보를 이용자 위험 없이 평가할 수 있는가	요청을 후보에도 복제해 보내되 응답은 champion 것만 돌려줌	5

표 10.1 이 장에서 실행하는 다섯 가지 확인

☐ 다섯 가지 중 셋째를 특히 숫자로 확인함

- ○ “같은 코드로 만들었다”는 진술은 줄바꿈 규약 하나만 달라도 참인 채로 남지만, 그 두 파일의 sha256은 다름
- ○ 그래서 배포에서는 같은 코드인지를 묻는 데서 멈추지 않고 바이트가 같은지까지 계산해 확인함
  - – 그 계산이 6절 [4]에서 숫자로 나타남

☐ 실행하기 전에 네 가지를 먼저 적음. 남이 만든 산출물을 먼저 보면 거기 적힌 값이 정답으로 보여 “왜 이 값인가”를 묻지 않게 됨

표 10.2 예측표 — 실행하기 **전에** 채운다

예측할 것	내 예측	실제	어긋났다면 원인
밤 배치가 죽은 다음 날, 낮의 조회 3건 중 오류로 끝나는 것은 몇 건인가			
요청 7건 중 모델에 닿기 전에 거절되는 것은 몇 건인가			
shadow를 켠 요청 6건 중 이용자가 후보 모델의 값을 받는 것은 몇 건인가			
줄바꿈만 바꾼 `app.py` 복제본은 원본과 sha256이 같겠는가			

☐ 이 장에서 실행할 코드는 넷이며 순서가 정해져 있음

순서	실행할 파일	하는 일	절	선행 조건
1	`10-2-batch-vs-online.py`	배치 추론과 온라인 추론의 실패 형태 대조	3	9주차 산출물(저장소 포함)
2	`10-3-serving-contract.py`	신원 응답과 입력 검증 계약 집행	4	9주차 산출물
3	`10-4-shadow-compare.py`	champion·challenger 병행 실행과 후보 재평가	5	9주차 산출물
4	`10-5-release-gate.py`	검수표 재계산과 바이트 동일성 대조	6	10주차 산출물(저장소 포함)
선택	`10-1-model-api/`	FastAPI·Docker로 실제 서빙 API 구동	4·6	mlflow·FastAPI·Docker

○ 1~4번은 9주차가 남긴 훈련 쌍과 이 장의 과거 실행 증거를 입력으로 읽음. 넷 다 서로 독립이므로 순서를 바꿔도 됨

○ 선택 실습만 패키지 설치와 Docker가 필요함

시작 전 준비

☐ 1~4번은 표준 라이브러리만 쓰므로 설치할 패키지가 없음(실습 검증 환경은 Python 3.13)

☐ 선택 실습은 mlflow·FastAPI·pytest를 설치해야 하고, 컨테이너 단계는 Docker 데몬이 켜져 있어야 함

○ 아래 한 줄이 Python 버전·패키지·Docker 데몬을 점검하고 가상환경을 준비함

```
python scripts/setup_practice.py 10
```

3. 배치와 온라인은 속도가 아니라 실패의 형태로 갈린다

☐ 모델이 있다고 예측이 이용자에게 닿는 것은 아님

○ 누군가 어느 시점에 모델을 실행해야 하고, 그 시점의 선택지가 둘임

○ **배치 추론(batch inference)** 은 예측을 미리 만들어 저장해 둠

– 매일 밤 자치구 전부의 익일 민원 건수를 한꺼번에 계산해 표에 넣어 두면, 낮의 조회는 계산이 아니라 저장된 값을 꺼내는 일이 됨

○ **온라인 추론(online inference)** 은 요청이 도착한 순간 계산함

– 이용자가 입력을 보내면 그 자리에서 모델을 실행해 답을 만듦

☐ 어느 쪽인지는 두 질문으로 갈림

○ 요청 시점에만 알 수 있는 입력이 필요한가

– 이용자가 방금 입력한 값이나 현재 위치처럼 미리 알 수 없는 입력이 있으면 배치로는 만들 수 없음

○ 예측이 쓰일 지점을 미리 열거할 수 있는가

– “서울 자치구의 내일 예측”처럼 대상이 유한하면 미리 다 만들어 둘 수 있고, 임의의 입력 조합이면 불가능함

□ 정작 중요한 것은 **깨졌을 때의 모양이 다르다**는 점임

○ 배치가 실패하면 오류가 한 건도 남지 않음

– 밤 배치가 멈춰도 낮의 조회는 어제 값을 오류 없이 돌려줌

– 화면을 보는 담당자는 낡은 예측을 최신 예측으로 읽음. 1주차의 침묵 실패가 예측 층에서 그대로 반복됨

○ 온라인이 실패하면 모든 요청이 즉시 실패함

– 모델을 못 불러오면 요청마다 오류가 나므로 아무도 모르고 지나갈 수 없음

○ 어느 쪽이 낫다고 정해져 있지 않고, 어느 쪽의 실패인지 알고 설계하는 것이 운영의 시작임

□ 첫 번째 코드를 실행함. 9주차가 남긴 훈련 쌍으로 같은 모델 둘을 다시 만들어 배치 표를 채움

```
cd practice/chapter10
python3 code/10-2-batch-vs-online.py
```

```
# 출처: practice/chapter10/results/10-2-batch-vs-online.log
[1] 입력 — 9장 훈련 쌍 6개, 자치구 3개
    지문 96f6b6b3ce9dcdd1… (9장 산출물과 동일)
    champion(평균 예측기) 훈련 y 평균 36/6 = 6.0
    challenger(선형회귀) 절편 3.6818  기울기 0.4091
    자치구별 전일 건수  어제(2026-07-02) {'11440': 5, '11620': 5, '11680': 6}  오늘
(2026-07-03) {'11440': 5, '11620': 5, '11680': 8}

[2] 배치 추론 — 밤에 자치구 전부를 미리 계산해 표로 저장
    champion    어제 표 {'11440': 6.0, '11620': 6.0, '11680': 6.0}
                오늘 표 {'11440': 6.0, '11620': 6.0, '11680': 6.0}
    challenger  어제 표 {'11440': 5.7273, '11620': 5.7273, '11680': 6.1364}
                오늘 표 {'11440': 5.7273, '11620': 5.7273, '11680': 6.9545}

[3] 오늘 밤 배치가 죽었다 — 낮의 조회 3건은 어제 표를 그대로 읽는다
    champion    조회 3건 전부 성공, 오류 0건, 응답 코드 200
                오늘 값과 어긋난 자치구 0개 (없음)
    challenger  조회 3건 전부 성공, 오류 0건, 응답 코드 200
                오늘 값과 어긋난 자치구 1개 {'11680': 0.8181}

[4] 온라인 추론이 같은 이유로 죽으면
    조회 3건 중 실패 3건 — 아무도 모르고 지나갈 수 없다

[5] 배치의 낡음을 조회가 판정하려면 — 표에 기준일을 함께 저장한다
    표의 기준일 2026-07-02 / 조회한 날 2026-07-03 → 낡음 판정 예
    기준일이 없으면 같은 조회 3건이 전부 정상으로 보인다
```

□ 화면에 나온 값에서 읽을 것

- ○ **배치 실패는 오류를 한 건도 만들지 않음** — 조회 3건이 전부 응답 코드 200으로 끝나고 오류는 0건임. 온라인 쪽은 같은 원인으로 3건 전부 실패함. 깨진 정도는 같은데 화면에 남는 흔적이 정반대임
- ○ **낡은 값이 얼마나 틀렸는지는 모델이 입력을 쓰는지에 달렸음** — champion은 평균 예측기여서 어제 표와 오늘 표가 똑같고 어긋난 자치구가 0개임. 입력을 쓰는 challenger로 바꾸면 강남구(11680)에서 0.8181이 어긋남
- ○ 그러므로 **champion 쪽은 낡음이 값으로는 절대 드러나지 않음.** 값을 비교해 낡음을 잡겠다는 계획은 이 모델에서 원리적으로 작동하지 않음
- ○ **기준일을 표에 함께 저장하면 판정이 생김** — 표의 기준일 2026-07-02와 조회한 날 2026-07-03이 다르므로 낡음이 값과 무관하게 판정됨. 조회가 값만 읽으면 이 판정 자체가 불가능함

> - **조회 날짜는 구성값이다**
> - 9주차 산출물에는 전일 건수와 당일 건수만 있고 “언제 조회했는가”는 없다. 어제와 오늘을 가르려고 두 날짜를 코드에 고정된 값으로 두었다.
> - 난수를 쓰지 않으므로 몇 번 실행해도 같은 값이 나오지만, 이 두 날짜는 실제로 관측된 조회 시각이 아니다. 예측값 자체는 9주차 훈련 쌍에서 계산한 값이다.

☐ 정직하게 덧붙이면 일 단위 민원 건수 예측은 배치로 충분함

- ○ 대상인 자치구는 열거할 수 있고 피처인 전일 건수는 하루 한 번 확정되므로 1분마다 새 예측을 요구할 이유가 없음
- ○ 그런데도 실습이 온라인 API를 세우는 이유는 관찰 대상 때문임 — 서빙 계약·컨테이너 포장·검증 게이트라는 배포의 규율이 온라인 서빙에서 한꺼번에 등장하기 때문임

☐ 거꾸로 배치로 충분한 문제에 상시 API를 세우면 비용과 장애 지점을 스스로 늘리는 과잉 설계가 됨

- ○ 발주 문서에서 “실시간 AI”라는 표현을 보면 요청 시점에만 아는 입력이 실제로 있는지부터 물어야 함

4. 서빙 API는 모델에 신원을 붙이고 입력 규격을 집행한다

☐ API는 모델을 호출 가능하게 만드는 데서 끝나지 않고, 모델이 지켜야 할 규격을 정해 집행함

- ○ 실습의 서빙 앱은 그 규격을 신원·예측·검증 셋으로 나눠 선언함

☐ **신원 응답** — 생존 확인 응답이 살아 있는지만 답하지 않고 무엇을 서빙 중인지를 함께 답함

- ○ 모델 이름과 버전, 그리고 그 모델을 어디서 읽어 왔는지를 돌려줌

☐ **신원이 실린 예측** — 예측 응답에 값만 싣지 않고 그 값을 만든 모델의 신원을 함께 실음

☐ **입력 검증** — 자치구 코드는 5자리 숫자, 전일 건수는 0 이상 정수라는 선언을 스키마가 집행함

- ○ 잘못된 입력이 모델까지 흘러가 그럴듯한 숫자로 되돌아오는 경로를 막는 것이 목적임

☐ 신원 응답 한 줄이 왜 계약에 들어가는지는 감사 상황에서 드러남

- ○ 콜센터 상담 지원 시스템이 “이 예측의 근거를 소명하라”는 민원을 받았다고 하자
- ○ 응답에 모델 이름과 버전이 실려 있으면 그 시각 그 답을 만든 모델이 특정되고, 9주차 레지스트리(registry)를 따라 실행 기록·훈련 데이터 지문·파라미터까지 거슬러 올라갈 수 있음
- ○ 응답에 예측값만 있으면 그 소급은 “그때 아마 이 버전이었을 것”이라는 추정에서 멈춤
- ○ **응답에 신원 세 필드가 있으면 그 시각의 모델까지 거슬러 올라갈 수 있고, 없으면 추정에서 멈춘다.**

☐ 두 번째 코드를 실행함. 같은 계약을 표준 라이브러리로 다시 집행해 요청 일곱 건을 통과시킴

```
python3 code/10-3-serving-contract.py
```

```
# 출처: practice/chapter10/results/10-3-serving-contract.log
[1] GET /health — 살아 있는가가 아니라 무엇을 서빙 중인가
    status           ok
    model_name       complaint_daily_forecaster
    model_version    1
    model_source     registry-alias
    shadow_enabled   True

[2] POST /predict — 요청 7건을 계약에 통과시킨다
    200  정상 — 강남구, 전일 9건        {'lawd_cd': '11680', 'x_prev_count': 9} →
forecast 6.0
    200  정상 — 마포구, 전일 0건        {'lawd_cd': '11440', 'x_prev_count': 0} →
forecast 6.0
    422  음수 건수                   {'lawd_cd': '11680', 'x_prev_count': -1}
         └ x_prev_count는 0 이상 정수여야 함
    422  코드 자리에 한글              {'lawd_cd': '강남구', 'x_prev_count': 9}
         └ lawd_cd는 5자리 숫자 문자열이어야 함
    422  필드 누락                   {'x_prev_count': 9}
         └ lawd_cd 필드 누락
    422  코드 4자리                  {'lawd_cd': '1168', 'x_prev_count': 9}
         └ lawd_cd는 5자리 숫자 문자열이어야 함
    200  형식은 맞지만 없는 자치구         {'lawd_cd': '99999', 'x_prev_count': 9} →
forecast 6.0
    통과 3건 / 거절 4건

[3] 스키마 계약이 보는 것과 보지 않는 것
    형식을 어겨 거절된 요청 4건
    형식은 맞지만 사전에 없는 자치구 코드가 통과한 요청 1건 — ['99999']

[4] 응답에 신원을 실을 때와 싣지 않을 때
    신원 있음 4개 필드 — forecast 6.0, complaint_daily_forecaster v1, 출처 registry-
alias
    신원 없음 1개 필드 — forecast 6.0
```

☐ 화면에 나온 값에서 읽을 것

- ○ **거절은 모델에 닿기 전에 일어남** — 요청 7건 중 4건이 422로 끝났고 그 4건은 모델을 한 번도 실행하지 않았음. 음수 건수·한글 코드·필드 누락·네 자리 코드가 전부 여기서 걸림
- ○ **전일 건수가 응답에 반영되지 않음** — 전일 9건과 전일 0건이 똑같이 6.0을 받음. 오류가 아니라 champion인 v1이 평균 예측기이기 때문이며, 훈련 y의 평균 36/6 = 6.0이 손으로 검산됨(`practice/chapter9/data/output/ch9_experiment_report.json`의 `training_pairs`)

- ○ **스키마는 값의 모양만 봄** — 존재하지 않는 자치구 코드 `99999`가 형식을 지켰다는 이유로 통과해 6.0을 받음. 그 코드가 실제로 있는지는 표준 코드 사전 (`practice/chapter6/data/input/lawd_cd_seoul.csv`)이 판정할 일이며, 스키마 계약이 대신해 주지 않음
- ○ **신원이 붙은 응답은 필드가 4개, 붙지 않은 응답은 1개임** — 두 응답의 예측값은 똑같이 6.0임. 감사 질의에 답할 수 있는 쪽은 앞의 응답뿐이고, 그 차이를 만드는 것은 계산이 아니라 세 필드임

□ 모델은 요청마다 로드하지 않고 기동 시점에 한 번만 로드함

- ○ 요청마다 로드하면 로드 시간과 레지스트리 장애가 요청 경로 안으로 들어오기 때문임
- ○ 무겁고 위험한 일은 기동 시점에 몰고 요청 경로에는 계산만 남기는 것임

□ 여기에 대가가 하나 붙음

- ○ 이미 떠 있는 프로세스는 별칭이 옮겨진 사실을 모름
- ○ 로드 코드가 버전 번호가 아니라 `models:/complaint_daily_forecaster@champion`처럼 별칭을 가리키므로 승격 때 코드는 한 줄도 바뀌지 않음

  – 그 대신 다시 기동하기 전까지는 옛 모델을 계속 서빙함

- ○ 다음 절의 두 층 문제가 여기서 나옴

5. shadow는 실패 반경을 0으로 두고 후보를 재평가한다

□ 새 버전이 옳다는 확신은 배포 전에 완전해지지 않음

- ○ 그래서 배포 전략은 전부 한 질문의 답이다 — **틀렸을 때 잘못된 응답을 받는 이용자가 전체의 몇 %인가(실패 반경, blast radius)를 어떻게 줄이는가.**

  – 전량을 한 번에 교체하면 반경은 100%임

  – 새 버전에 트래픽 일부만 흘리는 카나리(canary)는 그 비율만큼으로 반경을 줄임

□ 실습이 고른 것은 **shadow** 임

- ○ 실제 요청을 후보 모델에도 복제해 보내되 이용자에게 돌려주는 응답은 champion 것만 씀
- ○ 후보가 무엇을 답하든 이용자에게 닿지 않으므로 실패 반경이 0임
- ○ 대가는 계산 자원이 두 배로 들고 “이용자가 그 답에 어떻게 반응하는가”는 관찰할 수 없다는 점임

□ 공공에서 이 전략이 특히 맞는 이유가 하나 더 있음

- ○ 트래픽을 나눠 보내면 같은 시각에 시민마다 다른 모델이 응답하게 됨
- ○ 민원 배정이나 복지 판정처럼 권리에 영향을 주는 결정에서는 그 중간 상태 자체가 형평성 문제가 됨

☐ 세 번째 코드를 실행함. 같은 훈련 쌍으로 두 모델을 적합해 같은 요청에 병행 실행함

```
python3 code/10-4-shadow-compare.py
```

```
# 출처: practice/chapter10/results/10-4-shadow-compare.log
[1] 두 모델을 같은 훈련 쌍 6개로 적합
    champion   v1 평균 예측기      상수 6.0
    challenger v2 선형회귀        절편 3.6818  기울기 0.4091
    9장이 기록한 계수 {'intercept': 3.6818, 'slope': 0.4091} 와 일치: 예

[2] shadow 호출 — 훈련 쌍의 전일 건수를 그대로 요청 6건으로 보낸다
    자치구         전일      이용자가 받는 값        로그에만 남는 값        차이
    11440       5            6.0          5.7273  -0.2727
    11440       5            6.0          5.7273  -0.2727
    11620       5            6.0          5.7273  -0.2727
    11620       5            6.0          5.7273  -0.2727
    11680       6            6.0          6.1364   0.1364
    11680       8            6.0          6.9545   0.9545
    이용자에게 나간 응답 6건 전부 champion 값 — challenger는 한 건도 닿지 않았다

[3] 과거 실행이 남긴 shadow 관찰과 대조
    이번 계산  전일 9 → champion 6.0 / challenger 7.3636
    과거 실행  전일 9 → champion 6.0 / challenger 7.3636
    (출처 ch10_deploy_report.json — 일치: 예)
    두 값의 차이 1.3636 가 이 절의 관찰 자료다

[4] 로그만으로 후보를 재평가 — 요청 6건의 절대오차 평균
    champion   1.0
    challenger 1.0455
    9장 기록  champion 1.0 / challenger 1.0455
    후보가 더 낫지 않음 — 승격 보류 판정이 실요청 위에서도 유지된다

[5] challenger가 요청 처리 중 죽으면
    shadow 실패 1건 / 이용자에게 나간 응답 6건
```

☐ 화면에 나온 값에서 읽을 것

- ○ **후보 값은 한 건도 이용자에게 닿지 않음** — 요청 6건의 응답이 전부 champion의 6.0이고, challenger의 값은 로그 열에만 남음. 예측표에서 이 칸을 0이 아닌 값으로 적었다면 shadow와 카나리를 섞어 본 것임
- ○ **차이가 자치구마다 방향까지 다름** — 마포·관악에서는 후보가 0.2727 낮고 강남에서는 0.1364와 0.9545 높음. “후보는 대체로 더 크게 답한다” 같은 보정이 성립하지 않음

- ○ **로그만으로 승격 판정을 다시 할 수 있음** — 절대오차 평균이 champion 1.0, challenger 1.0455로 9주차 기록과 같음. 이용자 위험을 0으로 둔 채 반려된 후보를 실요청 위에서 재평가했고, 판정은 그대로 유지됨
- ○ **후보가 실패해도 응답은 전부 나감** — shadow 실패가 1건 생겼는데 이용자에게 나간 응답은 6건 그대로임. 실패 반경 0이 문장이 아니라 숫자로 확인된 것임

□ 교체를 실제로 하려면 층이 둘이라는 점을 알아야 함

- ○ **모델 층**의 교체는 레지스트리에서 champion 별칭이 v1에서 v2로 옮겨지는 일임
- ○ **서비스 층**의 교체는 서빙 프로세스를 다시 기동해 트래픽을 새 실물로 넘기는 일임
- ○ 앱이 기동 시점에 모델을 한 번만 로드하므로(4절) 별칭만 옮겨서는 서빙에 반영되지 않음
- ○ 두 층이 다 움직여야 교체가 완성되고, 되돌릴 때도 별칭을 되돌리고 이전 실물을 다시 기동하는 두 동작이 필요함

□ 되돌리는 **방법**을 아는 것과 **언제** 되돌릴지 정해 두는 것은 다른 일임

- ○ 사고가 커지는 이유는 대개 되돌릴 방법이 없어서가 아니라 되돌릴 기준이 없어서임
- ○ 지표가 나빠지는 동안 “조금 더 보자”가 반복되고, 되돌림은 잘못된 응답을 받은 이용자가 다 늘어난 뒤에 이루어짐

□ 그래서 롤백(rollback) 트리거는 **어떤 지표를, 어느 임계에서, 얼마의 관측 창으로** 판정할지 세 조각을 함께 적어야 함

- ○ 셋 중 하나라도 비어 있으면 되돌릴 시점이 정해지지 않으므로 트리거로 작동하지 않음
- ○ 이 장 범위에서 바로 쓸 수 있는 트리거는 배포 직후 스모크(smoke)임 — 전일 9건에 6.0이 나오지 않으면 이전 실물로 되돌림

□ 임계와 관측 창은 되돌림의 대가에서 역산함

- ○ 이전 이미지를 다시 띄우는 정도로 끝나면 창을 짧게 잡아 빨리 되돌리는 편이 쌈
- ○ 저장소 스키마 변경까지 따라오면 창을 늘려 오탐을 줄이는 편이 나음

6. 게이트는 층으로 쌓고, 동일성은 바이트로 확인한다

□ 이 실습을 만들면서 실제로 겪은 오류가 층의 필요성을 그대로 보여 줌

- ○ 서빙 앱은 설정 기본값을 자기 파일 위치에서 두 단계 위 디렉터리 기준으로 계산했음
- ○ 서버 없이 하는 테스트가 통과했고 호스트 실행도 통과했는데, 컨테이너에서는 생존 확인 응답이 끝내 나오지 않았음
- ○ 이미지 안에서 앱은 `/app/app.py`에 놓이므로 “두 단계 위”가 존재하지 않았고, 경로 계산이 파일을 읽는 시점에 실패했음

☐ 주목할 것은 수정 내용이 아니라 실패의 구조임

- ○ 앞의 두 게이트는 이 결함을 **원리적으로** 잡을 수 없었음 — 둘 다 저장소 배치 위에서 실행되므로 그 경로 가정이 항상 참이었기 때문임
- ○ 실행 환경이 달라야만 드러나는 결함이 있다는 것, 그래서 컨테이너 검증이 생략 가능한 중복이 아니라 독립 게이트라는 것이 이 사고의 교훈임

☐ 네 번째 코드를 실행함. 두 배치에서 같은 경로 식을 평가하고, 검수표를 과거 실행 증거에서 다시 계산함

```
python3 code/10-5-release-gate.py
```

```
# 출처: practice/chapter10/results/10-5-release-gate.log
[1] 층마다 잡는 결함이 다른 이유 — 같은 식이 배치에 따라 갈린다
    저장소 배치   /repo/practice/chapter10/code/10-1-model-api/app.py 상위 경로 5단계
parents[1] → /repo/practice/chapter10
    이미지 배치   /app/app.py                                          상위 경로 1단계  par-
ents[1] → IndexError: list index out of range

[2] 배포 전 검수표를 과거 실행 증거에서 다시 계산한다
    통과  1_자동_테스트_통과
          판정 근거 tests.passed  /  커밋된 검수표 True
    통과  3_알려진_입력_알려진_출력_스모크
          판정 근거 host.forecast  /  커밋된 검수표 True
    통과  5_컨테이너_호스트_동일_예측(환경_동등성)
          판정 근거 container.forecast == host.forecast  /  커밋된 검수표 True
    미달  6_모델_실물_이미지_고정_및_출처_동봉
          판정 근거 파일 존재 검사  /  커밋된 검수표 True  ← 불일치
    미달  8_검증_파이프라인_정의(CI_워크플로)
          판정 근거 파일 존재 검사  /  커밋된 검수표 True  ← 불일치
    측정값으로 판정하는 항목 6개 — 전부 재계산되어 같은 값이 나왔다
    파일 존재로 판정하는 항목 2개 — 이 저장소에 그 파일이 없어 불일치 2건

[3] 게이트 하나를 뒤집어 보면 — 컨테이너 예측이 6.1로 나왔다면
    통과 항목 6개 → 5개, 배포 판정 중단

[4] '같은 코드'와 '같은 바이트'
    원본           5708 바이트  줄 120개  sha256 1718607ffbd39aad…
    줄바꿈 변환본   5588 바이트  줄 120개  sha256 ec7ec65397a3194a…
    줄 단위 내용 동일: 예  /  바이트 동일: 아니오
```

☐ 위 발췌는 검수표 8항목 중 다섯을 옮긴 것이고, 2·4·7번도 같은 형식으로 전부 통과로 나옴

☐ 화면에 나온 값에서 읽을 것

- ○ **같은 코드 한 줄이 배치에 따라 갈림** — 저장소 배치에서는 상위 경로가 5단계라 두 단계 위가 존재하고, 이미지 배치에서는 1단계뿐이라 같은 식이 예외로 끝남. 앞 두 게이트가 이 결함을 못 잡은 이유가 여기 숫자로 나와 있음
- ○ **검수표 8항목 중 6항목이 측정값에서 그대로 재계산됨** — 테스트 통과 수·모델 버전·예측값·응답 코드·두 환경의 일치·레지스트리 버전이 과거 실행 증거만으로 다시 판정됨. 손으로 옮겨 적은 값이 아니라 계산된 값임
- ○ **나머지 2항목은 다른 기계에서 뒤집힘** — 6·8번은 파일이 있는지를 묻는데, 모델 export 디렉터리는 `.gitignore`가 제외했고 CI 워크플로 파일은 이 저장소에 없음. **판정 근거가 측정값이냐 파일 존재냐에 따라 검수표의 이식성이 갈린다**
- ○ **게이트 하나가 뒤집히면 배포 판정이 따라 바뀜** — 컨테이너 예측만 6.1로 가정하자 통과 항목이 6개에서 5개로 줄고 판정이 중단으로 바뀜. 손으로 쓰는 문서였다면 이 항목이 바뀌어도 문서는 그대로 남음
- ○ **줄 내용이 같은데 해시가 다름** — 줄바꿈 규약만 바꾼 복제본은 줄 수도 120개로 같고 줄 단위 내용도 같은데 바이트가 120개 줄고 sha256이 달라짐. “같은 코드로 만들었다”는 진술이 걸러 내지 못하는 차이가 이것임

> - **sha256 값 자체는 기계마다 다를 수 있다**
> - 위의 `1718607ffbd39aad…`는 이 저장소를 이 기계에 체크아웃했을 때의 `app.py` 해시다. git이 줄바꿈을 변환하도록 설정된 기계에서는 다른 값이 나온다.
> - 여기서 확인한 것은 특정 해시값이 아니라, 내용이 같아 보여도 바이트 동일성은 따로 판정된다는 사실이다.

□ 검증을 층으로 쌓는 이유는 층마다 잡는 결함이 다르기 때문임

- ○ 서버 없이 앱을 직접 호출하는 테스트는 신원 누락·검증 누락 같은 계약 위반을 초 단위에 잡음
- ○ 호스트에서 실제 프로세스로 띄우는 확인은 기동 절차·포트·레지스트리 연결의 결함을 잡음
- ○ 컨테이너에서 하는 확인은 이미지 배치·의존성 누락·경로 가정의 결함을 잡음

□ 층의 마지막 질문이 **환경 동등성** 임 — 같은 입력에 호스트와 컨테이너가 같은 답을 내는가

- ○ 아래는 이 강의 환경에서 Docker를 켤 수 없어 **과거 실행이 남긴 산출물에서 옮긴 값**임

```
# 출처: practice/chapter10/data/output/ch10_deploy_report.json
  "consistency": {
    "container_forecast": 6.0,
    "host_equals_container": true,
    "host_forecast": 6.0,
    "matches_ch9_forecast_6_0": true
  },
  "container": {
    "forecast": 6.0,
    "health": {
      "model_name": "complaint_daily_forecaster",
      "model_source": "local-dir",
      "model_version": "1",
      "shadow_enabled": false,
      "status": "ok"
    },
    "invalid_input_status": 422,
    "predict_status": 200
  },
  "host": {
    "forecast": 6.0,
    "health": {
      "model_name": "complaint_daily_forecaster",
      "model_source": "registry-alias",
      "model_version": "1",
      "shadow_enabled": true,
      "status": "ok"
    },
    "invalid_input_status": 422,
    "predict_status": 200
  },
  ...
  "tests": {
    "passed": 4
  },
```

☐ 화면에 나온 값에서 읽을 것

- ○ **같은 입력에 두 환경이 같은 답을 냄** — 호스트 6.0과 컨테이너 6.0이 같고, 잘못된 입력은 양쪽 다 422로 거절함
- ○ **그런데 출처를 증명하는 방식이 다름** — 호스트는 `registry-alias`로 레지스트리에 물어서 버전을 답하고, 컨테이너는 `local-dir`로 이미지 안에 함께 넣어 둔 파일을 읽어 답함. 검수는 "같은 답이 나오는가"와 "무엇을 서빙 중인지 답할 수 있는가"를 둘 다 물음
- ○ **컨테이너에서는 shadow가 꺼져 있음** — `shadow_enabled`가 호스트에서 참이고 컨테이너에서 거짓임. 컨테이너는 레지스트리를 보지 않으므로 challenger 별칭을 읽을 수 없기 때문이며, 폐쇄망에서도 돌아가게 만든 대가임

☐ 컨테이너 쪽 출처가 파일이 될 수 있는 이유는 이미지가 코드와 의존성 버전과 **모델 실물까지** 하나의 불변 단위로 포장하기 때문임

- ○ 9주차 재현 조건 가운데 “환경”이 문서가 아니라 실행 가능한 실물로 고정되고, 그 결과 이미지가 곧 되돌림 단위가 됨
- ○ 빌드 절차도 내려받는 데서 그치지 않고 내려받은 모델을 로드해 예측까지 시켜 본 뒤 어느 별칭의 몇 번 버전이었는지를 함께 동봉함(같은 파일의 `bootstrap.export.export_check_forecast` = 6.0)
- ○ 쓰기가 성공했다는 사실을 믿지 않고 읽어서 확인하는 9주차의 규율이 그대로 이어진 것임

☐ 이 게이트들을 매번 실행하는 주체를 사람에서 저장소로 옮긴 것이 CI(Continuous Integration, 지속적 통합)다

- ○ CI는 새 검증을 더하는 수단이 아니고 게이트는 이미 다 있음
- ○ 바뀌는 것은 “한 줄 고쳤는데 뭘”이라며 건너뛸 선택지가 구조적으로 사라진다는 점뿐임

☐ 반대로 CI가 잡지 못하는 것도 분명함

- ○ 데이터가 변해 모델이 낡아 가는 드리프트는 11주차의 일임
- ○ **CI 통과는 배포 가능의 필요조건이지 충분조건이 아니다.**

7. 핵심 요약

☐ **1. 배치와 온라인은 속도가 아니라 깨졌을 때의 모양으로 갈린다.** (3절)

- ○ 밤 배치가 죽은 다음 날 조회 3건이 전부 응답 코드 200으로 끝나고 오류가 0건이었으며, 온라인은 같은 원인으로 3건 전부 실패했다.
- ○ champion은 평균 예측기여서 어제 표와 오늘 표가 같아 어긋난 자치구가 0개였고, 입력을 쓰는 challenger로 바꾸자 강남구에서 0.8181이 어긋났다. 값 비교로 낡음을 잡겠다는 계획은 모델에 따라 원리적으로 작동하지 않는다.

☐ **2. 서빙 API의 최소 계약은 신원·예측·검증 셋이다.** (4절)

- ○ 요청 7건 중 4건이 모델에 닿기 전에 422로 거절됐고, 통과한 3건은 전일 건수와 무관하게 6.0을 받았다.
- ○ 신원이 붙은 응답은 필드가 4개, 붙지 않은 응답은 1개이고 예측값은 똑같이 6.0이다. 감사 질의에 답할 수 있는 쪽은 앞의 응답뿐이다.
- ○ 다만 스키마는 값의 모양만 보므로 존재하지 않는 코드 `99999`가 통과했다. 코드의 실재 여부는 표준 코드 사전(`practice/chapter6/data/input/lawd_cd_seoul.csv`)이 판정한다.

☐ **3. shadow는 실패 반경을 0으로 두고 후보를 재평가한다.** (5절)

- ○ 요청 6건의 응답이 전부 champion의 6.0이었고 challenger 값은 로그에만 남았다. 후보가 처리 중 죽은 1건에서도 이용자 응답 6건은 그대로 나갔다.
- ○ 로그만으로 절대오차 평균을 다시 세니 champion 1.0, challenger 1.0455로 9주차 판정이 실요청 위에서도 유지됐다.
- ○ 교체는 별칭 이동(모델 층)과 재기동(서비스 층) 두 층이 다 움직여야 완성되고, 되돌림에는 지표·임계·관측 창 세 조각이 필요하다.

☐ **4. 층마다 잡는 결함이 다르므로 검증은 겹쳐야 한다.** (6절)

- ○ 같은 경로 식이 저장소 배치(상위 5단계)에서는 성립하고 이미지 배치(상위 1단계)에서는 예외로 끝났다. 앞 두 게이트는 이 결함을 원리적으로 잡을 수 없었다.
- ○ 호스트와 컨테이너가 같은 입력에 6.0으로 같은 답을 냈지만 출처를 증명하는 방식은 `registry-alias`와 `local-dir`로 달랐고, 컨테이너에서는 shadow가 꺼져 있었다.

☐ **5. 검수표는 계산되어 나와야 하고, 동일성은 바이트로 확인한다.** (6절)

- ○ 검수표 8항목 중 6항목이 과거 실행 증거에서 그대로 재계산됐고, 컨테이너 예측만 6.1로 가정하자 통과 항목이 6개에서 5개로 줄어 판정이 중단으로 바뀌었다.
- ○ 나머지 2항목은 파일 존재로 판정하므로 파일이 없는 기계에서 뒤집혔다. 판정 근거를 측정값에 두어야 검수표를 다른 기계에서도 그대로 쓸 수 있다.
- ○ 줄바꿈만 바꾼 복제본은 줄 내용이 같은데도 sha256이 달랐다. “같은 코드”는 이 차이를 걸러 내지 못한다.

8. 과제

☐ 실습 넷을 실행해 만든 산출물 네 개를 LMS에 제출한다

1. `practice/chapter10/data/output/ch10_batch_vs_online.json`
2. `practice/chapter10/data/output/ch10_serving_contract.json`
3. `practice/chapter10/data/output/ch10_shadow_compare.json`
4. `practice/chapter10/data/output/ch10_release_gate.json`

☐ 이어서 네 가지를 각각 세 문장 이내로 쓴다

1. 본인 산출물 `ch10_batch_vs_online.json`의 `batch_failure.errors`와 `online_failure.errors`를 그대로 옮겨 적고, 두 값이 다른 것이 왜 배치 쪽이 더 위험하다는 뜻인지 쓴다.
2. `ch10_serving_contract.json`의 `summary`에서 `rejected_422`와 `accepted_but_unknown_district`를 나란히 적고, 스키마 계약이 막아 준 것과 막아 주지 못한 것을 구분해 쓴다.
3. `ch10_shadow_compare.json`의 `reevaluation`과 `blast_radius`를 옮겨 적고, 후보를 승격하지 않기로 한 판단의 근거가 이 두 값에서 어떻게 나오는지 쓴다.

4. 저장소에 포함된 `practice/chapter10/data/output/ch10_deploy_report.json`에서 `consistency.host_forecast`·`consistency.container_forecast`와 두 환경의 `health.model_source`를 옮겨 적고, 같은 답이 나왔는데도 출처를 증명하는 방식이 다른 이유를 쓴다.

☐ 표 10.2의 예측 네 칸에 실측을 나란히 채우고, 어긋난 칸마다 원인을 한 줄로 적어 함께 제출한다

☐ 값을 지어내지 않는다. 실행이 안 되면 어디서 막혔는지 그대로 쓴다

☐ 수업일 포함 7일 이내에 제출한다

9. 더 알아보기

☐ 실습을 더 해 보려면 `practice/chapter10/`에서 다음을 시도함

- ○ `10-2-batch-vs-online.py`의 배치 표를 champion 대신 challenger로만 만들어, 낡은 표의 오차가 어느 자치구에서 먼저 눈에 띄는지 확인함
- ○ `10-3-serving-contract.py`의 `CASES`에 요청을 더 넣어, 어떤 입력이 스키마를 통과해 모델까지 가는지 찾음
- ○ `10-4-shadow-compare.py`에서 champion과 challenger를 맞바꿔 보고, 이용자가 받는 값과 재평가 결론이 어떻게 달라지는지 확인함
- ○ `10-5-release-gate.py`의 검수표에서 파일 존재로 판정하는 두 항목을 측정값 판정으로 바꿔 보고, 다른 기계에서도 같은 결과가 나오는지 따져 봄
- ○ 패키지와 Docker를 설치할 수 있으면 `10-1-model-api/`의 서빙 API를 직접 띄워 6절의 컨테이너 값을 본인 기계에서 다시 만듦

☐ 뒤 주차에서 이어지는 내용

- ○ 배포한 모델이 데이터 변화로 낡아 가는 것을 어떻게 잡는지는 11주차에서 실행함
- ○ 피처와 모델의 변경을 누가 승인하고 어디에 남기는지는 12주차, 이 장의 서빙을 다른 구성요소와 묶는 통합은 13주차, 언어 모델 운영은 14주차에서 다룸

☐ 원문을 직접 보고 싶다면 다음 두 편이 출발점임

- ○ Google Cloud, 「MLOps: Continuous delivery and automation pipelines in machine learning」 — 배포 자동화 단계 구분. https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning
- ○ Martin Fowler, 「BlueGreenDeployment」 — 무중단 교체와 되돌림의 원형. https://martinfowler.com/bliki/BlueGreenDeployment.html